

PSX Stock Buy / Sell Signal Prediction System

Using Machine Learning on Pakistan Stock Exchange Market Data

Submitted by

Muhammad Abdullah 25K-7636

Rabia 25K-7612

Sheeraz 25K-7624

Submitted in partial fulfilment of the requirements for the course in Machine Learning

Submission Date: June 4, 2026

Table of Contents

Table of Contents	1
2. Abstract	3
3. Introduction	3
3.1 Background of Stock-Market Prediction	3
3.2 Importance of Buy/Sell Signal Generation	3
3.3 Motivation	3
4. Problem Statement	4
5. Project Objectives.....	4
5.1 Main Objective.....	4
5.2 Secondary Objectives.....	4
6. Literature Review	4
6.1 Stock-Prediction Techniques	4

6.2 Technical Indicators and the RSI.....	5
6.3 Machine Learning for Financial Forecasting.....	5
7. System Architecture.....	5
8. Methodology	6
8.1 Data Collection.....	6
8.2 Data Cleaning and Preprocessing.....	6
8.3 Feature Engineering	7
8.4 RSI Calculation Methodology	7
8.5 Target Definition (Labelling).....	8
8.6 Model Training and the Time-Based Split	8
8.7 Signal Generation.....	8
8.8 Backtesting and Risk Management	9
9. Team Contributions	10
10. Implementation Details	10
10.1 psx_scraper.py — Data Acquisition	10
10.2 clean_existing_data.py — Cleaning	10
10.3 rsi_calculator.py — Technical Indicators	10
10.4 random_forest.py and light_gbm.py — Modelling.....	10
10.5 run_pipeline.py — Orchestration	11
10.6 app.py — Streamlit Dashboard	11
11. Machine Learning Models.....	11
11.1 Random Forest.....	11
11.2 LightGBM	11
11.3 Why These Models Were Selected	11
11.4 Advantages and Limitations	12
12. Results and Analysis	12
12.1 Dataset Statistics.....	12
12.2 Signals Generated	12
12.3 Backtest Performance	13
12.4 Position-Sensitivity Analysis.....	14
12.5 Exit-Reason Analysis.....	15
13. Dashboard Overview	15
14. Challenges and Solutions	16

15. Future Enhancements	17
16. Conclusion.....	17
17. References	17

2. Abstract

This report presents the design, implementation and evaluation of an end-to-end machine-learning system that generates **Buy** and **Sell** trading signals for equities listed on the Pakistan Stock Exchange (PSX). Five years of daily market data — **749,460** records spanning **3,768** unique tickers (15 March 2021 to 13 March 2026) — were scraped automatically from the official PSX data portal, cleaned, and enriched with technical indicators including the Relative Strength Index (RSI) at multiple horizons, MACD, Average True Range (ATR), Bollinger Bands and moving-average distances. Two supervised classifiers — a **Random Forest** and a **LightGBM** gradient-boosting model — were trained on a strict time-based split (approximately four years of training data and the most recent twelve months held out for out-of-sample testing) to estimate the probability of a short-term upward price move. Model probabilities are combined with quantitative confirmation filters and a regime/risk screen to produce HOLD, BUY and STRONG BUY signals, which are then evaluated by a risk-managed portfolio backtest using ATR-based trailing stops and a hard loss cap. On the held-out year the Random Forest strategy returned **+139.3%** and the LightGBM strategy **+14.8%** on a starting capital of Rs. 100,000. All results are surfaced through an interactive seven-page Streamlit dashboard with a paper-trading simulator. The system is intended for educational and research purposes and not as financial advice.

3. Introduction

3.1 Background of Stock-Market Prediction

Forecasting the direction of financial markets is one of the most studied and most difficult problems in applied data science. The Efficient Market Hypothesis argues that prices already reflect all available information, yet decades of research and practice show that short-term, exploitable patterns do appear — especially in emerging markets where information disseminates unevenly. Two broad analytical traditions exist: **fundamental analysis**, which values a company from its financial statements and macroeconomic context, and **technical analysis**, which infers future price behaviour from historical price and volume patterns. This project takes the technical-analysis route and augments it with machine learning, allowing the model to learn non-linear relationships between many indicators simultaneously.

3.2 Importance of Buy/Sell Signal Generation

For an active investor the central question is not merely *where will the price go?* but *should I act now?* A signal-generation system answers the second question directly by emitting a discrete, timestamped recommendation per stock. Converting a continuous probability into an explicit BUY/SELL/HOLD decision — and pairing it with predefined risk controls — turns a predictive model into an actionable, testable trading strategy whose performance can be measured objectively.

3.3 Motivation

Retail participants on the PSX frequently trade on rumour, sentiment and informal tips, with little access to the systematic, data-driven tooling common in developed markets. There is no clean public price API, and

few free analytical platforms target PSX specifically. This project is motivated by the opportunity to build a fully automated, reproducible pipeline — from raw data collection through to a decision dashboard — that demonstrates how modern machine learning and disciplined risk management can be applied to a frontier market in a transparent and explainable way.

4. Problem Statement

Individual investors lack a systematic, repeatable and risk-aware method for deciding when to buy or sell PSX-listed stocks. The available historical data is not exposed through a convenient API and contains noise, derivative instruments and formatting inconsistencies that make direct analysis difficult. Furthermore, naïve predictive models tend to over-fit historical prices and ignore transaction risk. The problem addressed by this project is therefore three-fold:

- **Data:** reliably acquire and clean several years of PSX daily market data without an official API.
- **Prediction:** learn, from engineered technical indicators, the probability that a stock will rise by a meaningful margin within a short horizon, validated honestly on unseen data.
- **Decision & risk:** convert those probabilities into explainable Buy/Sell signals and verify their economic value through a risk-managed backtest, rather than on accuracy alone.

5. Project Objectives

5.1 Main Objective

To develop a machine-learning-based system that generates Buy and Sell signals for PSX stocks using historical market data, technical indicators and predictive models, and to evaluate the economic performance of those signals through a realistic, risk-controlled backtest.

5.2 Secondary Objectives

- Automate the collection of multi-year PSX historical data via web scraping.
- Clean, de-duplicate and normalise the raw dataset for reliable downstream analysis.
- Engineer a robust set of technical features, including multi-period RSI, MACD, ATR and Bollinger Bands.
- Train and compare two complementary models — Random Forest and LightGBM — under an identical, leakage-free evaluation protocol.
- Design a transparent signal-generation framework combining model probability with quantitative filters.
- Implement a portfolio backtester with explicit position sizing, trailing stops and loss caps.
- Deliver an interactive Streamlit dashboard for exploration, comparison and paper trading.

6. Literature Review

6.1 Stock-Prediction Techniques

Approaches to stock-movement prediction broadly progress from classical statistical models such as ARIMA and GARCH, through classical machine-learning methods (Support Vector Machines, decision trees and ensemble methods), to modern deep-learning architectures such as Long Short-Term Memory (LSTM) networks and Transformers. Patel et al. (2015) demonstrated that tree-ensemble methods using technical

indicators can predict the direction of Indian indices with competitive accuracy, while Fischer and Krauss (2018) showed that LSTM networks can extract profitable signals from S&P 500 constituents. Tree-based ensembles remain popular for tabular financial features because they are robust, require little scaling, and resist over-fitting better than unregularised models.

6.2 Technical Indicators and the RSI

Technical indicators compress raw price and volume into interpretable signals. The Relative Strength Index (RSI), introduced by J. Welles Wilder (1978), is a bounded momentum oscillator between 0 and 100 that measures the magnitude of recent gains relative to recent losses; values above 70 conventionally indicate overbought conditions and below 30 oversold conditions. Other widely used indicators include the Moving Average Convergence Divergence (MACD), the Average True Range (ATR) for volatility, and Bollinger Bands for relative price extremes. Murphy (1999) provides the canonical practitioner reference for these tools. This project computes RSI at 1-, 7- and 30-day horizons together with the indicators above.

6.3 Machine Learning for Financial Forecasting

Breiman (2001) introduced the Random Forest, an ensemble of decorrelated decision trees that reduces variance through bagging and random feature selection. Ke et al. (2017) proposed LightGBM, a histogram-based gradient-boosting framework that grows trees leaf-wise and scales efficiently to large datasets. Both are strong baselines for tabular problems and are used here precisely because financial feature matrices are tabular, noisy and class-imbalanced. The scikit-learn library (Pedregosa et al., 2011) provides the Random Forest implementation used in this work.

7. System Architecture

The system follows a layered, file-decoupled architecture orchestrated end-to-end by a single script, `run_pipeline.py`. Each layer writes intermediate artefacts (CSV files) that the next layer consumes, so components can be developed, tested and re-run independently. Figure 1 shows the five architectural layers and their constituent modules.

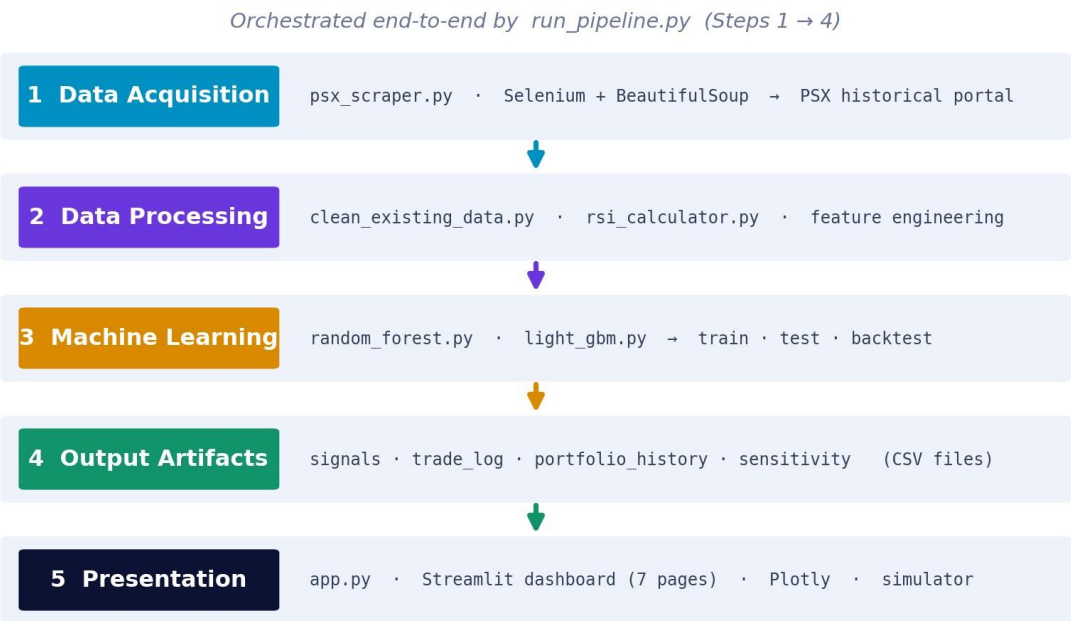


Figure 1. Five-layer system architecture of the PSX prediction system, orchestrated by `run_pipeline.py`.

The **Data Acquisition** layer scrapes the PSX historical portal. The **Data Processing** layer cleans the data and computes technical features. The **Machine Learning** layer trains both models, generates signals and runs the backtest. The **Output Artifacts** layer persists signals, trade logs, portfolio histories and sensitivity analyses to CSV. Finally, the **Presentation** layer — the Streamlit application — reads those artefacts and renders the interactive dashboard. Figure 2 shows the same process as a linear workflow.



Figure 2. End-to-end processing workflow, from the PSX data source to the Streamlit dashboard.

8. Methodology

8.1 Data Collection

Historical data is collected by `psx_scraper.py`, which automates a headless-capable Chrome browser using **Selenium** (with `webdriver_manager` for driver management) and parses each day's results table with **BeautifulSoup** and `pandas.read_html`. The scraper iterates over every business day in a five-year window

(`pandas.bdate_range`), injects the target date into the portal's date field via JavaScript, triggers the search, and paginates through the results. A 'smart wait' compares the first cell of the table between page clicks to confirm that the next page has actually loaded before scraping it, which avoids duplicated or missed rows.

Each day's pages are concatenated and de-duplicated on the symbol column before being appended to the master dataset, and a `Date` column is attached. The raw output is saved to `psx_historical_data_clean.csv`.

8.2 Data Cleaning and Preprocessing

Cleaning is performed by `clean_existing_data.py`. Derivative and futures contracts on the PSX use a hyphenated symbol convention (for example `TRG-MAR`); these rows are removed so that only ordinary cash-market equities remain. Numeric columns that the portal exports as strings containing thousands-separators (commas) are stripped and coerced to numeric types, and rows missing an essential field (symbol, date or close) are dropped. This step reduced the dataset from 749,460 raw rows to **463,378** clean rows.

Listing. Removing hyphenated (derivative) symbols in `clean_existing_data.py`.

```
dataset_cleaned = dataset[~dataset['SYMBOL'].str.contains('-', na=False)].copy()
```

8.3 Feature Engineering

Within both model scripts, features are computed per symbol (grouped by ticker, sorted by date) so that indicators never leak across stocks. Table 1 summarises the engineered indicators and the eight features ultimately supplied to the classifiers.

Table 1. Engineered technical indicators and the eight features used for model training.

Indicator	Definition / Formula	Model feature(s)
RSI (1,7,30 d)	Wilder momentum oscillator on close price	RSI_7D, RSI_30D
MACD	EMA(12) – EMA(26); signal = EMA(9); histogram = MACD – signal	MACD_Hist
ATR (14 d)	Mean of True Range over 14 days; ATR% = ATR / close	ATR_Pct
Bollinger Bands	SMA(20) $\pm$ 2 σ ; width = (upper – lower) / SMA(20)	BB_Width
EMA / SMA	Distance of close from EMA(20) and SMA(200)	Dist_EMA20, Dist_SMA200
Volume	Volume relative to its 20-day moving average	Vol_Ratio

8.4 RSI Calculation Methodology

The RSI is computed in `rsi_calculator.py`. For each symbol the close series is forward-filled and differenced; positive differences form the gain series and the magnitudes of negative differences form the loss series. Average gain and average loss are smoothed with an exponentially weighted moving average using a smoothing factor $\alpha = 1/n$ (Wilder-style smoothing), and the index is obtained as $RSI = 100 - 100/(1+RS)$, where $RS = \text{AvgGain}/\text{AvgLoss}$. When the average loss is zero the RSI is set to 100. Figure 3 illustrates the computation, which is repeated for $n \in \{1, 7, 30\}$.



Periods $n \in \{1, 7, 30\} \rightarrow RSI_{1D}, RSI_{7D}, RSI_{30D}$ (if AvgLoss = 0 then RSI = 100)

Figure 3. RSI computation pipeline for each stock, producing RSI_1D, RSI_7D and RSI_30D.

Listing. Core RSI computation (rsi_calculator.py).

```

gain = delta.clip(lower=0) loss = -1 * delta.clip(upper=0) avg_gain =
gain.ewm(alpha=1/period, min_periods=period, adjust=False).mean() avg_loss
= loss.ewm(alpha=1/period, min_periods=period, adjust=False).mean() rs =
avg_gain / avg_loss rsi = np.where(avg_loss == 0, 100, 100 - (100 / (1 +
rs)))

```

8.5 Target Definition (Labelling)

The prediction is framed as binary classification: will the stock rise by a target margin within a target horizon? The two models deliberately use different, complementary definitions. The Random Forest predicts a rise of more than 1.5% within 5 trading days; LightGBM predicts a stronger move — more than 3% within 10 trading days — to better match swing-trading execution.

Listing. Forward-looking binary targets for the two models.

```
# Random Forest target
group['Target_5D'] = (group['CLOSE'].shift(-5) >
group['CLOSE'] * 1.015).astype(int) # LightGBM target
group['Target_10D'] = (group['CLOSE'].shift(-10) > group['CLOSE'] * 1.03
).astype(int)
```

8.6 Model Training and the Time-Based Split

To prevent look-ahead bias, the data is split chronologically rather than randomly: the most recent twelve months are held out for testing and all earlier data (about four years) is used for training. Models are fitted on the eight features, probabilities are produced for the test period via `predict_proba`, and out-of-sample quality is reported with the ROC-AUC statistic. Both models use balanced class weights because positive (up-move) labels are the minority class. Figure 4 depicts the training and prediction pipeline.

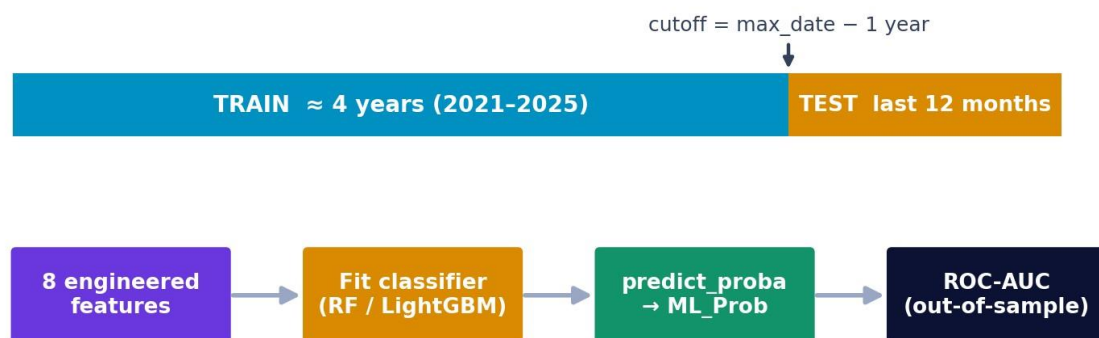


Figure 4. Machine-learning training and prediction pipeline with a chronological train/test split.

8.7 Signal Generation

A raw probability is not treated as a trade. As shown in Figure 5, a candidate must pass a funnel of filters: a probability gate, a set of quantitative trend/momentum/volume confirmations, a liquidity-and-volatility regime filter, and an overbought rejection. A stock earns a BUY when the probability gate and at least two confirmations are satisfied, and a STRONG BUY when the higher probability threshold and three confirmations are met; otherwise it remains HOLD. Table 2 lists the exact thresholds per model.

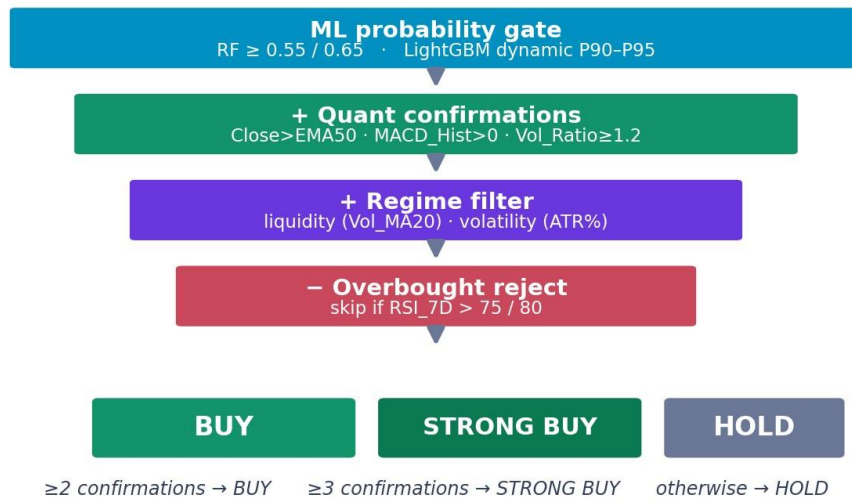


Figure 5. Signal-generation decision funnel converting model probability into a graded trade signal.

Table 2. Signal-generation thresholds for each model (from the generate_signals functions).

Criterion	Random Forest	LightGBM
Probability gate (BUY)	≥ 0.55	dynamic 90th pct, floor 0.45
Probability gate (STRONG BUY)	≥ 0.65	dynamic 95th pct, floor 0.50
Trend confirmation	Close > EMA(50)	Close > EMA(50) > SMA(200)
Momentum confirmation	MACD histogram > 0	MACD histogram > 0
Volume confirmation	Vol_Ratio ≥ 1.2	Vol_Ratio ≥ 1.2
Regime filter	Vol_MA20 > 25,000 and ATR% > 0.01	Vol_MA20 > 50,000 and ATR% > 0.015
Overbought reject	RSI_7D > 75	RSI_7D > 80

8.8 Backtesting and Risk Management

Signals are evaluated by an event-driven portfolio simulator starting from

Rs. 100,000. Each day, open positions are checked against their exit rules, then new BUY/STRONG BUY candidates are entered subject to a maximum-positions limit (the default pipeline uses 2). The Random Forest engine uses equal capital allocation with a 2.5×ATR trailing stop, a hard 10% loss cap, a 4×ATR reward target and a 30-day time stop. The LightGBM engine uses volatility-based position sizing (2% account risk per trade, capped at 12% of equity), a 2×ATR trailing stop, an 8% loss cap, a 3×ATR reward target, a break-even lock once a position gains 4%, and a 15-day time stop. Both engines additionally sweep the maximum-positions limit from 2 to 10 to produce a sensitivity analysis.

9. Team Contributions

The work was divided by module, with clear ownership while integration was coordinated by the team lead. Table 3 summarises each member's responsibilities and the source files they authored.

Table 3. Team member responsibilities and module ownership.

Member	Role	Modules / Files	Responsibilities
Muhammad Abdullah	Team Lead	<code>psx_scraper.py</code> , <code>rsi_calculator.py</code> , <code>random_forest.py</code> , <code>light_gbm.py</code>	Scraped PSX historical data; built the RSI calculator; implemented and trained the Random Forest and LightGBM models; integrated and managed the overall pipeline.
Sheeraz	Data Engineer	<code>clean_existing_data.py</code>	Performed data cleaning, preprocessing and transformation; prepared datasets for feature engineering and model training.
Rabia	Dashboard Developer	<code>app.py</code> , <code>run_pipeline.py</code>	Developed the Streamlit dashboard and user interface; implemented visualization and the pipeline orchestration / deployment components.

10. Implementation Details

The project is implemented entirely in Python (pandas, NumPy, scikit-learn, LightGBM, Selenium, BeautifulSoup, Streamlit and Plotly). The key modules and their responsibilities are described below, with representative code listings.

10.1 `psx_scraper.py` — Data Acquisition

Drives Selenium/Chrome to the PSX historical portal, iterates business days, paginates each day's table, de-duplicates by symbol, and writes the raw dataset. It also contains a helper to strip hyphenated symbols.

10.2 `clean_existing_data.py` — Cleaning

A focused, reusable cleaning routine that loads the raw CSV, removes hyphenated symbols and prints a before/after row count for auditability before saving the cleaned file.

10.3 `rsi_calculator.py` — Technical Indicators

Computes the RSI per symbol at 1-, 7- and 30-day horizons (Listing in Section 8.4) and appends the columns to produce `psx_with_rsi.csv`. The logic is exposed as an `add_rsi(input, output)` function so the pipeline can call it directly.

10.4 `random_forest.py` and `light_gbm.py` — Modelling

Each script loads the RSI-enhanced data, computes the technical feature set, performs the chronological split, trains its classifier, generates signals, runs the risk-managed backtest, prints a performance report, and exports signals, trade log, portfolio history and a position-sensitivity CSV. The model definitions are shown below.

Listing. Random Forest configuration (`random_forest.py`).

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=6,  
                                class_weight='balanced', random_state=42, n_jobs=-1)
```

Listing. LightGBM configuration (light_gbm.py).

```
lgbm_model = lgb.LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
max_depth=6, min_child_samples=20, subsample=0.8,
colsample_bytree=0.8, class_weight='balanced',
reg_alpha=0.1, reg_lambda=0.1, random_state=42)
```

10.5 run_pipeline.py — Orchestration

A single entry point that validates the raw data and runs the four pipeline stages in order, passing the shared hyperparameters (Rs. 100,000 capital, two maximum positions, 100 Random Forest trees, 300 LightGBM estimators).

Listing. Four-stage orchestration in run_pipeline.py.

```
clean_dataset(RAW_DATA, CLEAN_DATA)           # Step 1 - clean
add_rsi(CLEAN_DATA, RSI_DATA)                  # Step 2 - RSI
run_random_forest_model(RSI_DATA, ...)         # Step 3 - Random Forest
run_lightgbm_model(RSI_DATA, ...)              # Step 4 - LightGBM
```

10.6 app.py — Streamlit Dashboard

Loads the model output CSVs through a cached loader, computes comparison metrics per model, and renders seven interactive pages (described in Section 13). It applies a custom dark 'glassmorphism' theme and uses Plotly for all charts.

11. Machine Learning Models

11.1 Random Forest

A Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data and a random subset of features. Predictions are averaged across trees, which lowers variance and improves generalisation. Here it serves as a robust, interpretable baseline (100 trees, maximum depth 6).

11.2 LightGBM

LightGBM is a gradient-boosting framework that builds trees sequentially, each correcting the residual errors of the previous ensemble. Its histogram-based, leaf-wise growth and built-in L1/L2 regularisation give it a higher accuracy ceiling and excellent scalability on large tabular datasets (300 estimators, learning rate 0.05).

11.3 Why These Models Were Selected

Both models are state-of-the-art for tabular data, require minimal feature scaling, natively handle non-linear feature interactions, and expose class-weighting to address imbalance. Using one bagging model and one boosting model provides a meaningful comparison between two different ensemble philosophies on the same features and evaluation protocol. Table 4 contrasts their configurations.

Table 4. Hyperparameter comparison of the two classifiers.

Parameter	Random Forest	LightGBM
Algorithm	Bagged decision trees	Gradient-boosted trees
Estimators	100	300
Max depth	6	6

Learning rate	—	0.05
Num leaves	—	31
Regularisation	class_weight = balanced	L1 = 0.1, L2 = 0.1, subsample = 0.8
Target	+1.5% within 5 days	+3% within 10 days
Random state	42	42

11.4 Advantages and Limitations

- **Advantages:** robust to noise and outliers, little preprocessing required, capture non-linear interactions, fast to train, and provide feature-importance interpretability.
- **Limitations:** tree ensembles cannot model long temporal dependencies the way recurrent networks can; they assume the engineered features capture the relevant signal; and, like all back-tested strategies, performance can be regime-dependent and may not persist out-of-distribution.

12. Results and Analysis

12.1 Dataset Statistics

The cleaned dataset retains 463,378 daily records across 486 actively-traded symbols in the test window (14 March 2025 to 13 March 2026). Figure 6 shows row retention through the pipeline.

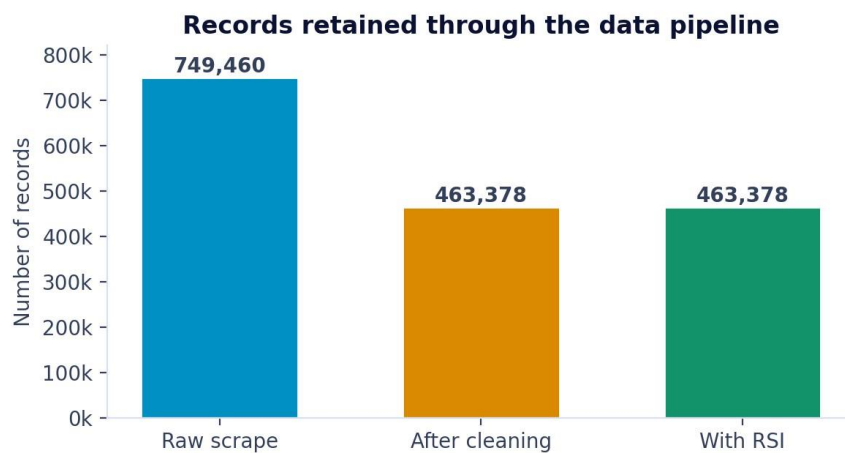


Figure 6. Record counts retained at each stage of the data pipeline.

12.2 Signals Generated

Over the test year the framework was deliberately selective. The Random Forest issued 319 BUY signals (and no STRONG BUY under its stricter triple-confirmation rule), while LightGBM issued 706 BUY and 9 STRONG BUY signals; the overwhelming majority of stock-days remained HOLD (Figure 7).

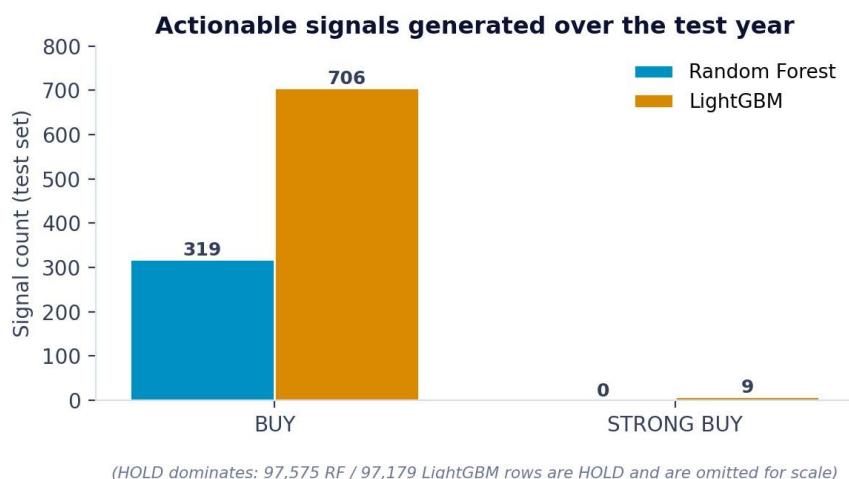


Figure 7. Actionable BUY / STRONG BUY signals generated by each model over the test year.

12.3 Backtest Performance

Table 5 reports the headline out-of-sample results, and Figure 8 plots the corresponding equity curves. With identical starting capital and a two-position limit, the Random Forest strategy grew Rs. 100,000 to Rs. 239,322 (+139.3%), while LightGBM reached Rs. 114,829 (+14.8%). The Random Forest's higher reward target (4×ATR) allowed winners to run further, producing a higher average trade return despite a comparable win rate.

Table 5. Out-of-sample backtest results (max 2 positions, Mar 2025 – Mar 2026).

Metric	Random Forest	LightGBM
Final equity	Rs. 239,322	Rs. 114,829
Net profit	Rs. 139,322	Rs. 14,829
Portfolio ROI	+139.3%	+14.8%
Trades executed	39	51
Win rate	46.2%	43.1%
Average trade ROI	+5.3%	+1.9%
Best / worst trade	+48.2% / -18.2%	+46.4% / -10.0%

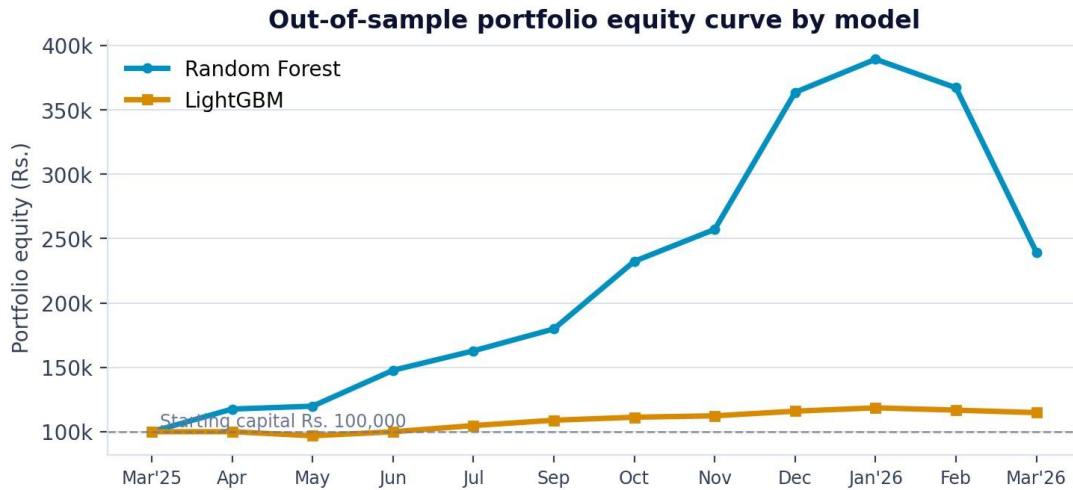


Figure 8. Out-of-sample portfolio equity curves for both models on Rs. 100,000 starting capital.

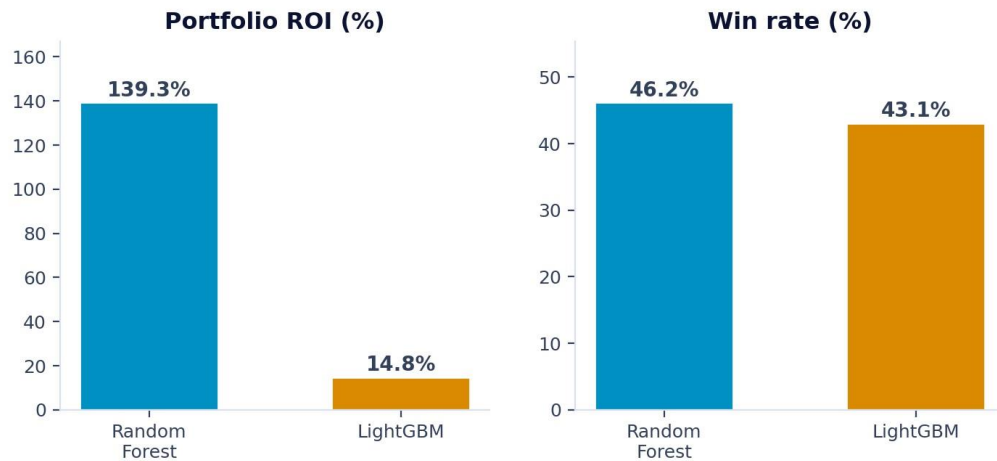


Figure 9. Portfolio ROI and win rate by model.

12.4 Position-Sensitivity Analysis

Because the maximum-positions limit strongly affects capital concentration, both engines were re-run for limits of 2 to 10 (Figure 10). The Random Forest peaks near +144% ROI at three positions and then declines as capital is spread into weaker signals, whereas LightGBM improves steadily with more slots, reaching roughly +83% at nine. This shows the two strategies have different optimal capacities.

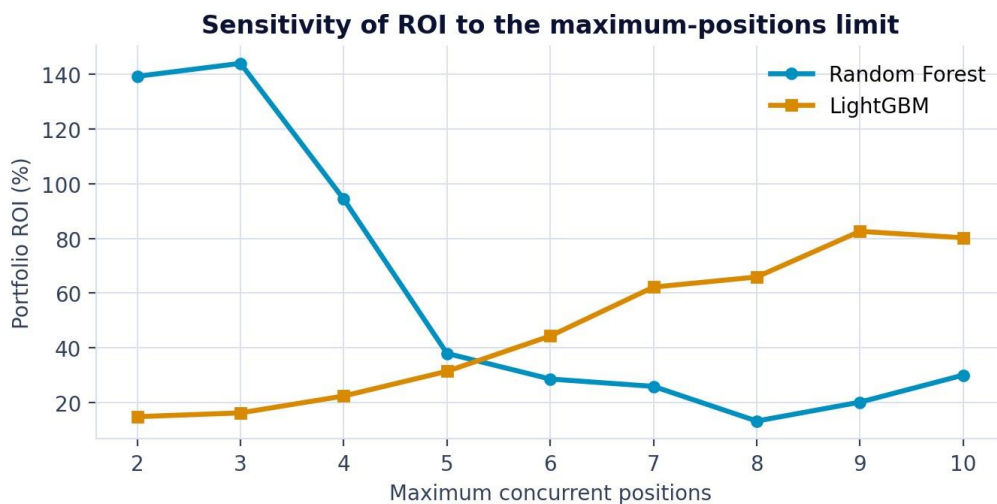


Figure 10. Portfolio ROI as a function of the maximum-positions limit.

12.5 Exit-Reason Analysis

Figure 11 breaks down why positions were closed. For the Random Forest, reward-target and trailing-stop exits together outnumber hard-stop losses, indicating the risk rules let profitable trades mature. LightGBM closes a larger share of trades on stop-losses, consistent with its more conservative outcome.

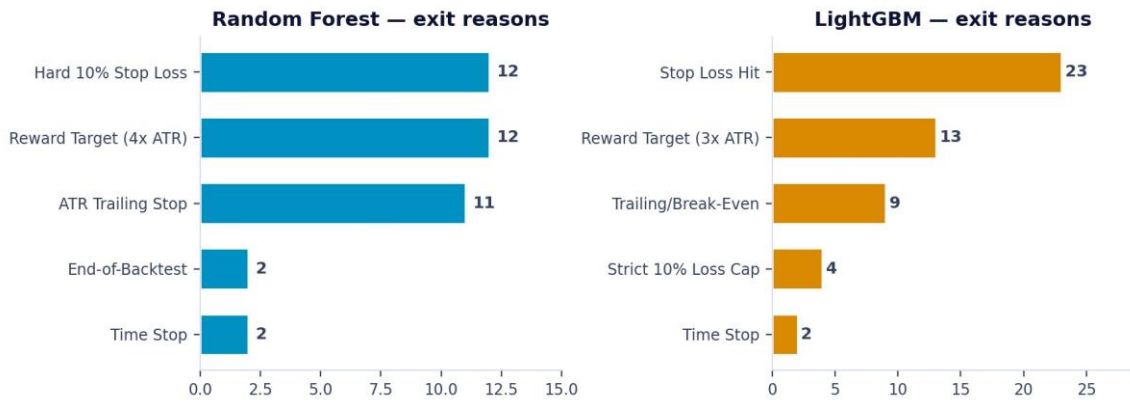


Figure 11. Distribution of trade exit reasons for each model.

13. Dashboard Overview

The Streamlit application in `app.py` is the user-facing layer. It never re-runs the models; instead it reads the exported CSV artefacts through a cached loader (`@st.cache_data`), computes comparison metrics, and presents seven interactive pages (Table 6). A representative mock-up is shown in Figure 12.

Table 6. The seven pages of the Streamlit dashboard.

Page	Function
Dashboard	KPI cards, portfolio equity curve, model scoreboard and signal mix
Model Comparison	Side-by-side metrics, metric bar charts and probability distributions
Position Sensitivity	ROI / win-rate versus the maximum-positions limit
Recommendations	Filterable BUY calls per date with cross-model overlap analysis
Simulator	Two-stock paper-trading account with a 10% stop and 8% target rule
Trade History	Filterable trade log with exit-reason distribution
Search	Free-text search across historical signals and trades

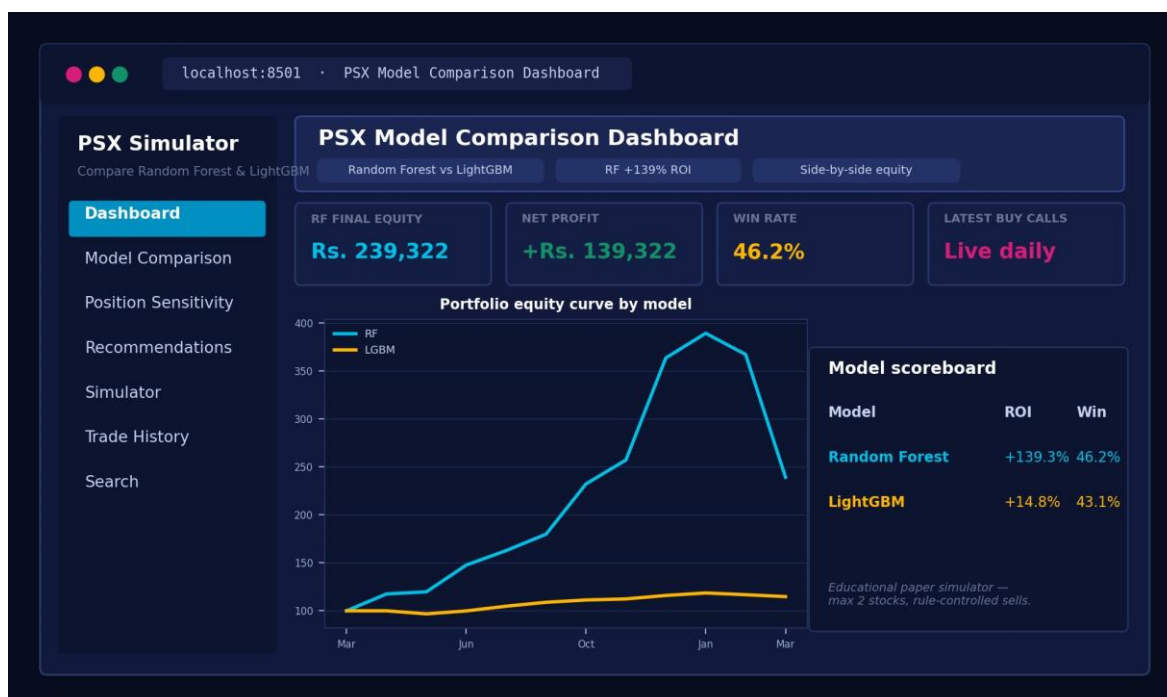


Figure 12. Mock-up of the interactive Streamlit dashboard recreated from app.py, with live equity curves.

The paper-trading simulator enforces the project's rule of holding at most two stocks at a time and a rule-controlled sell (a 10% stop loss or an 8% profit target). All charts are rendered with Plotly and the interface uses a custom dark theme for readability.

14. Challenges and Solutions

Table 7. Key technical challenges and how they were resolved.

Challenge	Solution adopted
No public price API	Automated Selenium + BeautifulSoup scraper against the PSX historical portal.
Paginated, asynchronously-loaded tables	A 'smart wait' that compares the first cell between page clicks before scraping.
Duplicate and derivative rows	De-duplication by symbol and removal of hyphenated derivative tickers.
String-formatted numbers	Comma stripping and numeric coercion during loading.
Look-ahead bias / over-fitting	Strict chronological train/test split with out-of-sample ROC-AUC reporting.
Class imbalance	Balanced class weights in both classifiers.
Down-side trading risk	ATR trailing stops, hard 8–10% loss caps, reward targets and time stops.

15. Future Enhancements

- Adopt **walk-forward** (rolling-origin) validation in place of a single hold-out year to test stability.
- Incorporate **transaction costs, slippage and liquidity** constraints for more realistic returns.
- Explore **deep-learning sequence models** (LSTM / Transformer) to capture temporal dependencies.
- Add a **live data feed** with scheduled refresh and automated signal alerts (email / messaging).
- Enrich the feature set with **fundamental and news-sentiment** data.
- Containerise and deploy the dashboard to the cloud for multi-user access.

16. Conclusion

This project delivered a complete, automated and reproducible machine-learning pipeline for generating Buy/Sell signals on the Pakistan Stock Exchange. Five years of data covering 3,768 tickers were scraped, cleaned and enriched with technical indicators; two complementary models — Random Forest and LightGBM — were trained under a leakage-free protocol; their probabilities were converted into explainable, risk-managed signals; and the results were validated through a portfolio backtest and surfaced in an interactive dashboard. On the held-out year the Random Forest strategy achieved a +139.3% portfolio ROI and LightGBM +14.8%, demonstrating that disciplined feature engineering and risk control can extract value from a frontier market. The principal limitation is that results are historical and exclude trading frictions; the future enhancements above outline a clear path toward a more rigorous and deployable system. The project should be regarded as an educational and research demonstration rather than financial advice.

17. References

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [2] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
- [3] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- [4] Murphy, J. J. (1999). *Technical Analysis of the Financial Markets*. New York Institute of Finance.
- [5] Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock priceindex movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268.
- [6] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Pakistan Stock Exchange. *Historical Data Portal*. <https://dps.psx.com.pk/historical>
- [9] The pandas development team (2020). *pandas-dev/pandas: Pandas*. <https://pandas.pydata.org>
- [10] Streamlit Inc. (2024). *Streamlit Documentation*. <https://docs.streamlit.io>
- [11] Plotly Technologies Inc. (2015). *Collaborative data science*. <https://plotly.com>